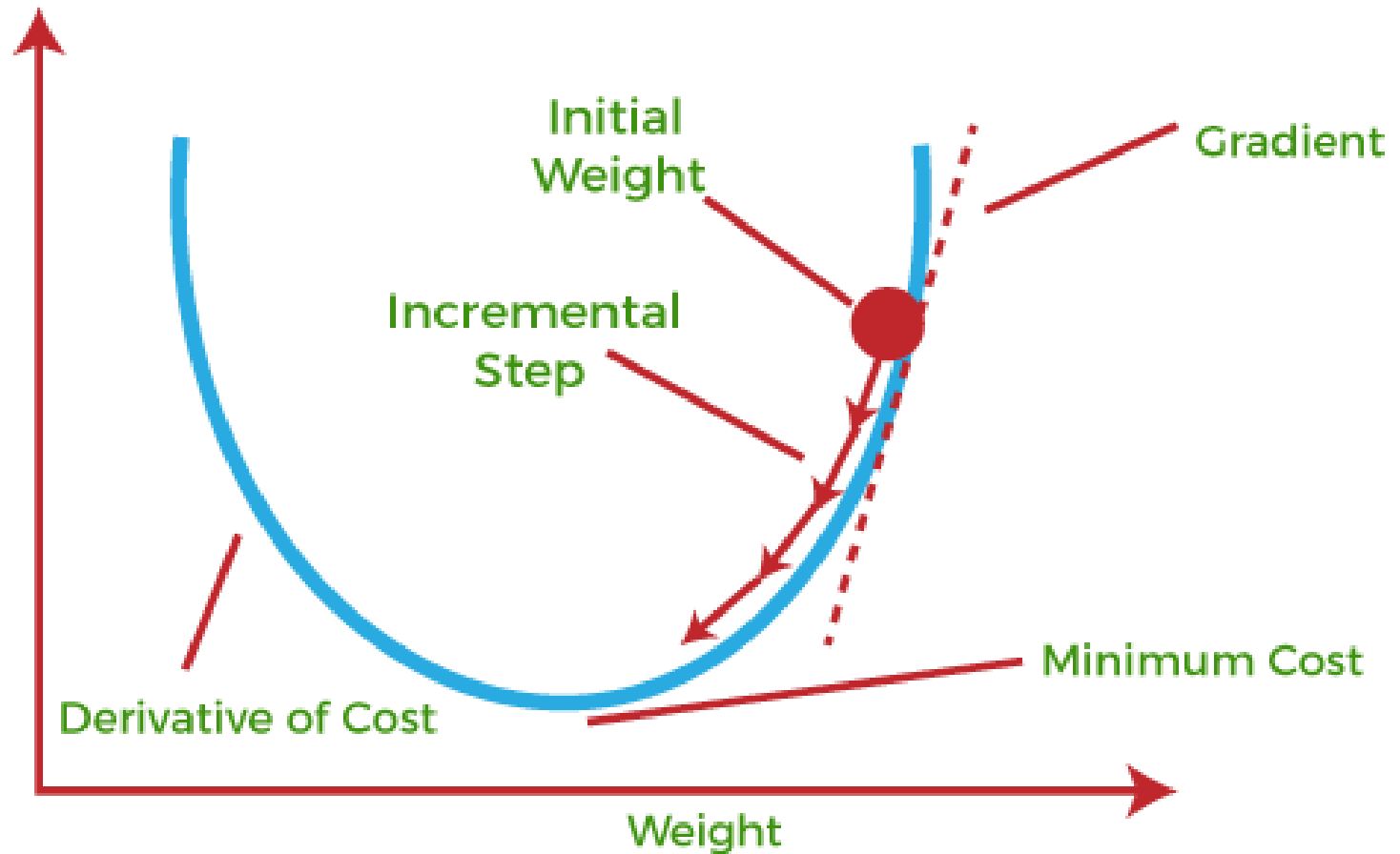


Gradient descent(경사하강)



경사하강법이란?

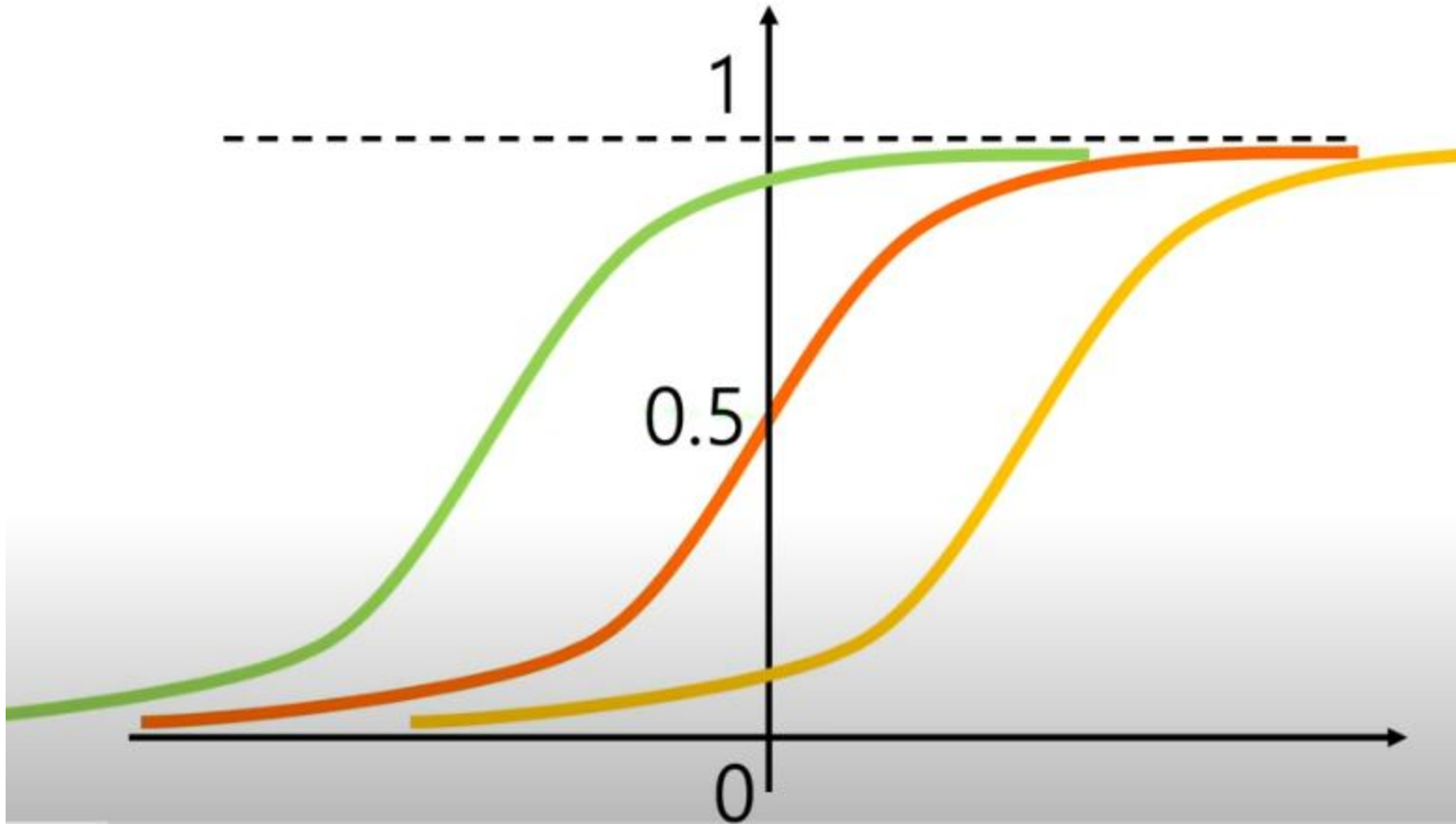
“ 주어진 손실함수에서 모델의 파라미터의 최적의 값을 찾는 머신러닝과 딥러닝의 최적화 알고리즘의 하나이다”

핵심 목표: 모델의 예측값과 실제 값의 차이(오차)를 나타내는 손실 함수를 최소화하는 **최적의 가중치(W)와 편향(b)**을 찾는 것입니다.

* 파라미터: 연결가중치 또는 편향

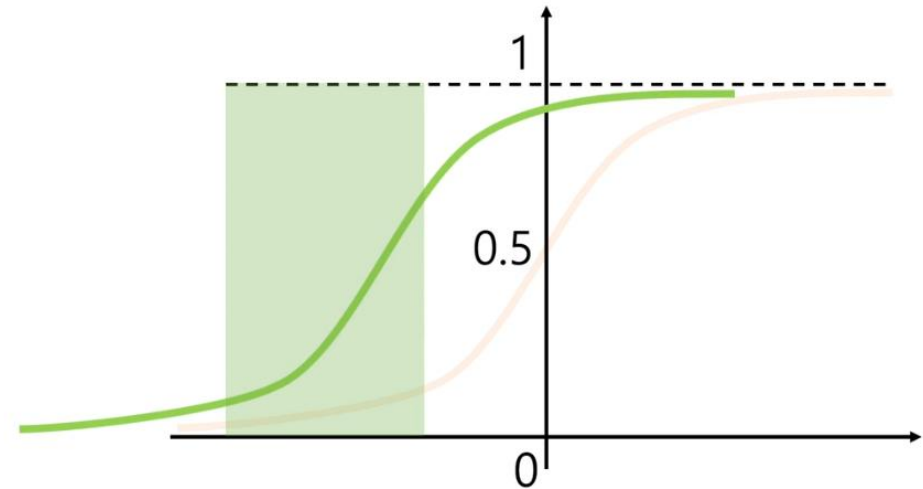
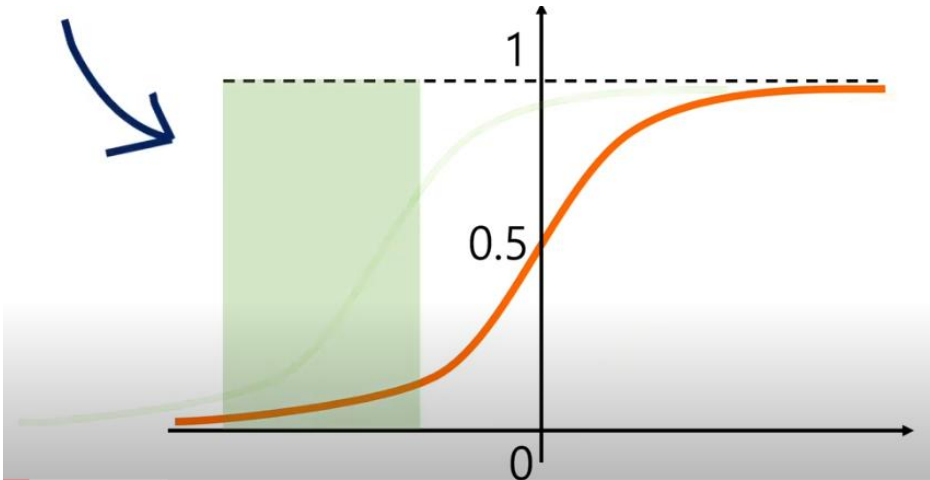
편향

- 활성화 함수를 좌측 또는 우측으로 이동: 신경망 학습 효율성 증가

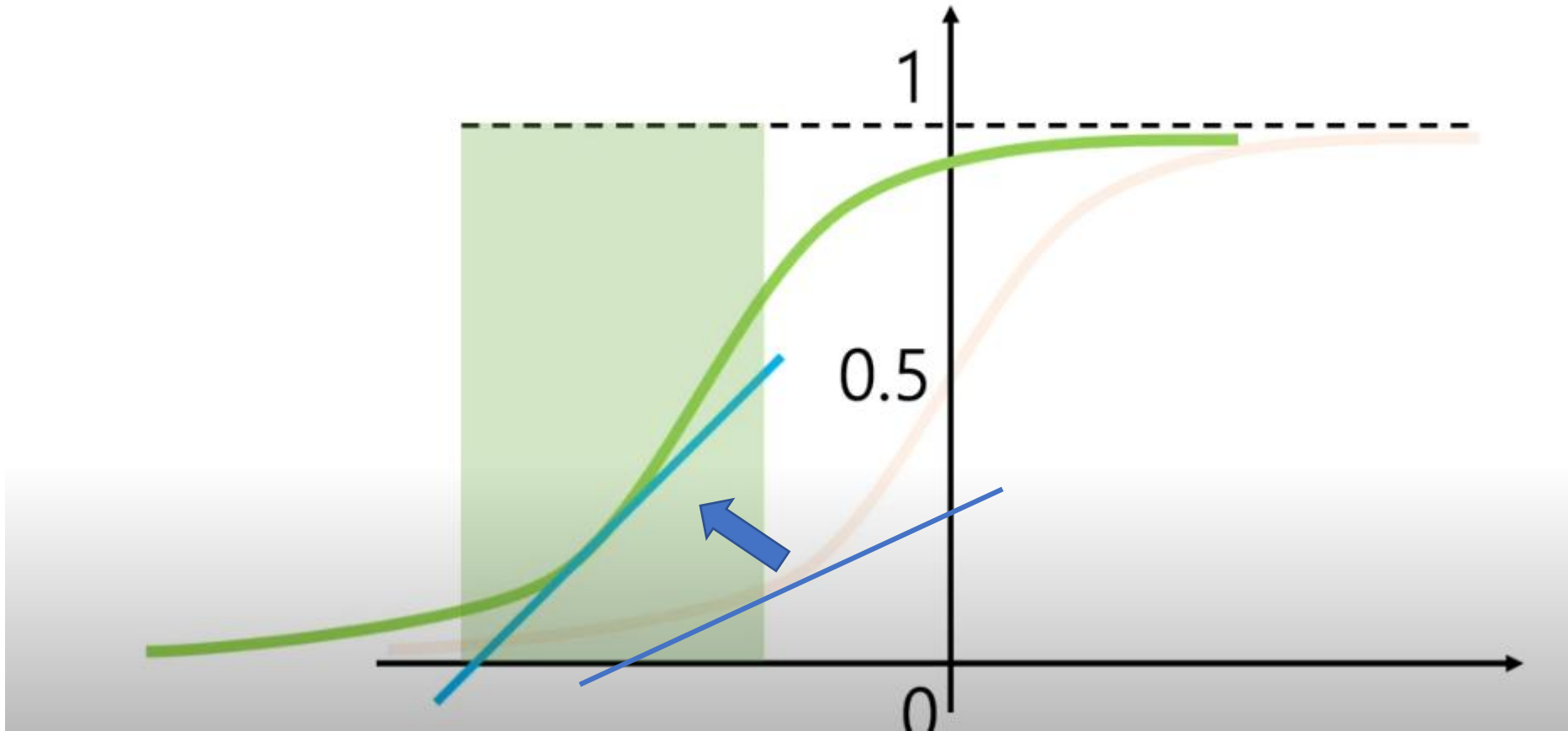


활성화 함수 입력값 : 좌측 쏠림

<시그모이드 함수 좌측 이동>



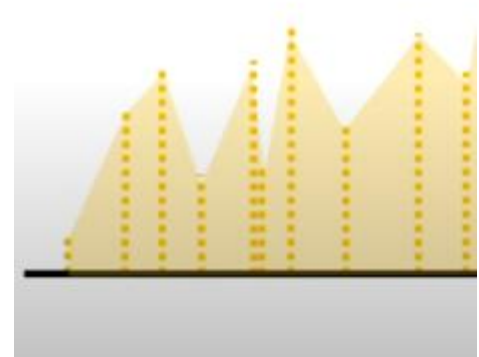
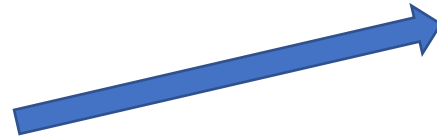
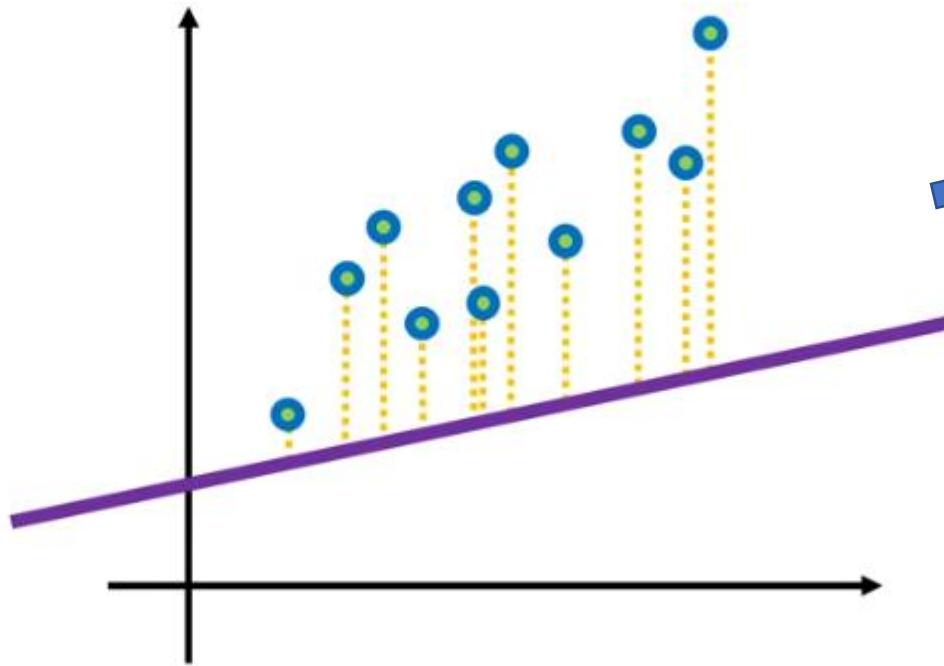
편향의 효율성 : 학습이 빨라짐(기울기 커짐)



왜 손실함수가 필요한가?

<손실함수, 오차와 유사한 개념>

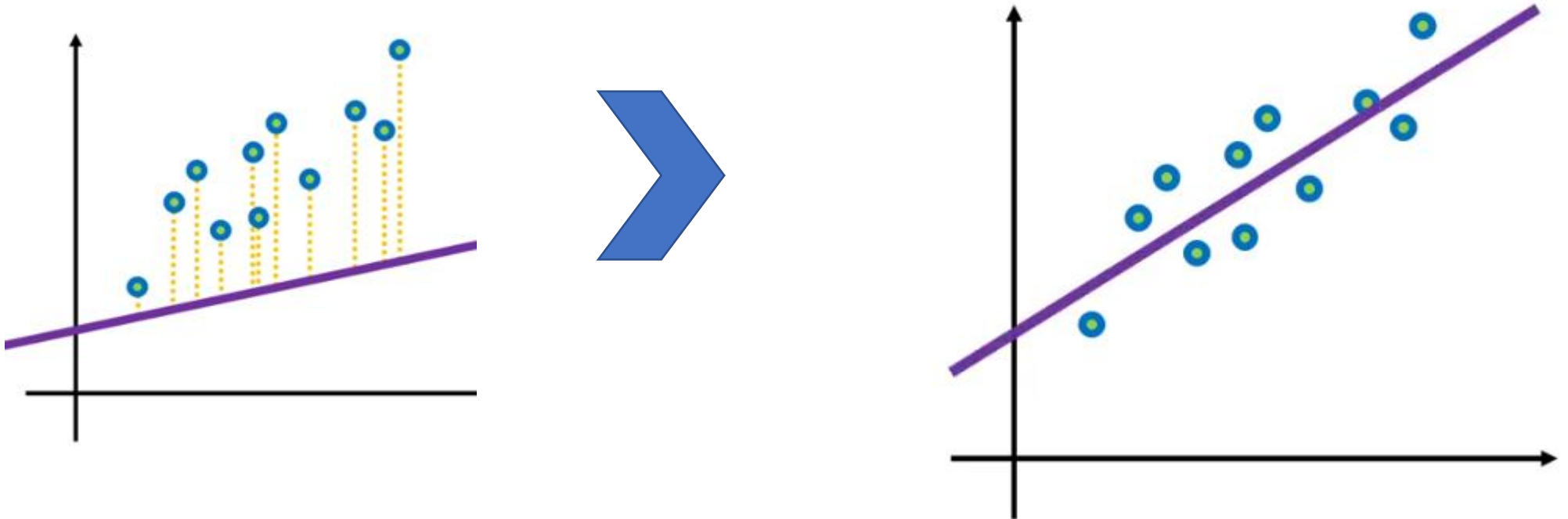
신경망 모델의 예측 값과 실제 값 간의 차이를 측정하는 함수입니다



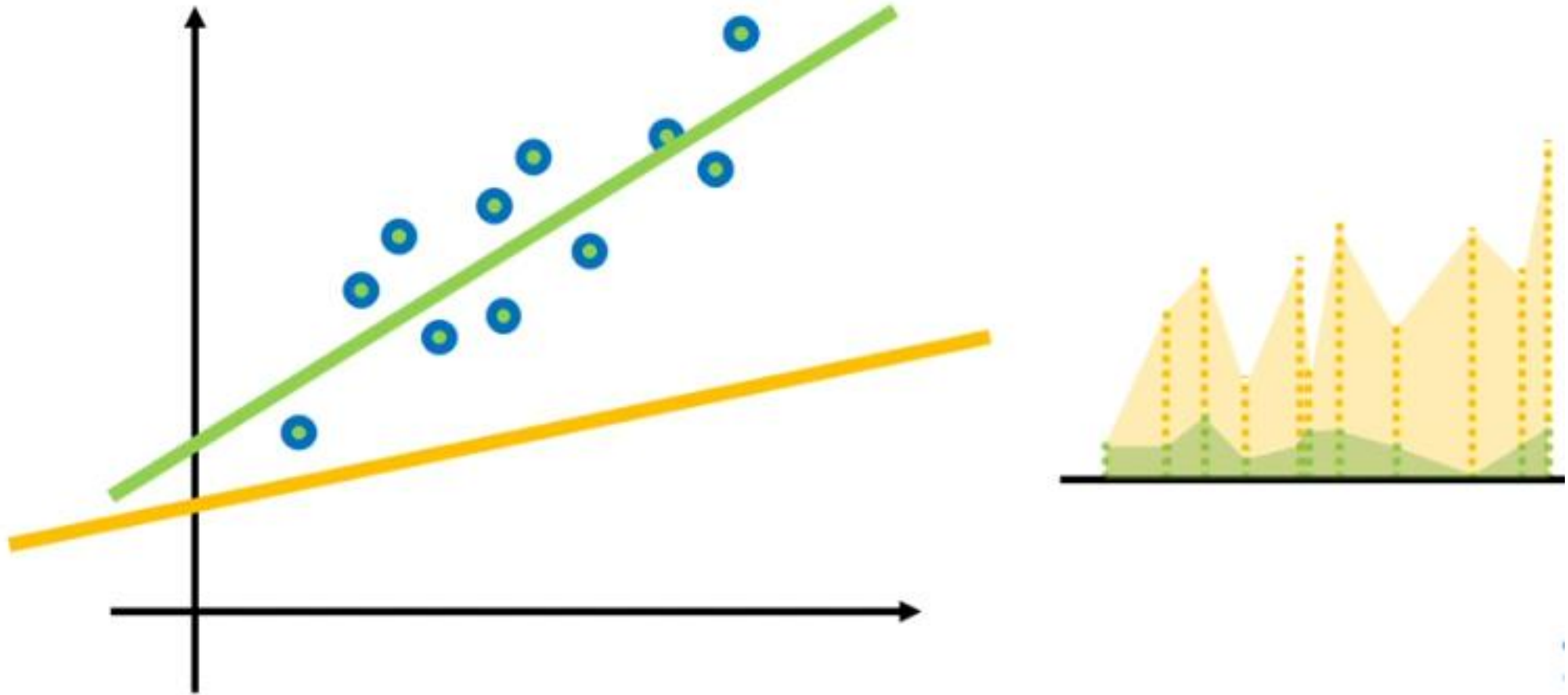
<오차의 합: 면적>

학습이란?

- 모든 파라미터(가중치, 편향 등)를 반복적으로 조금씩 튜닝하는 것



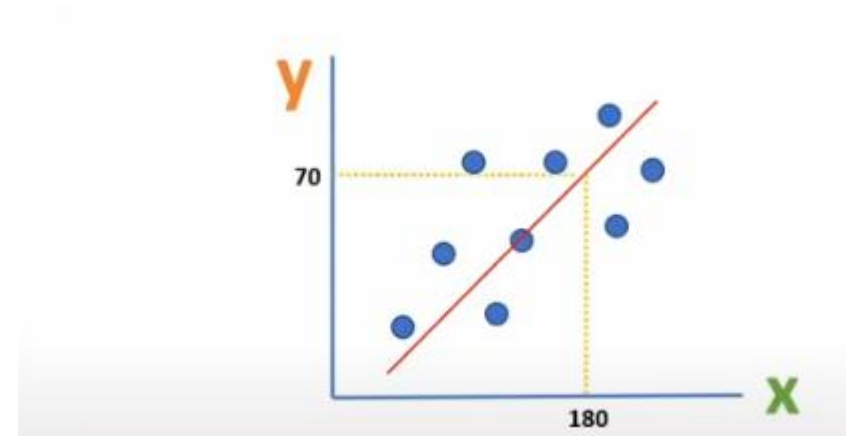
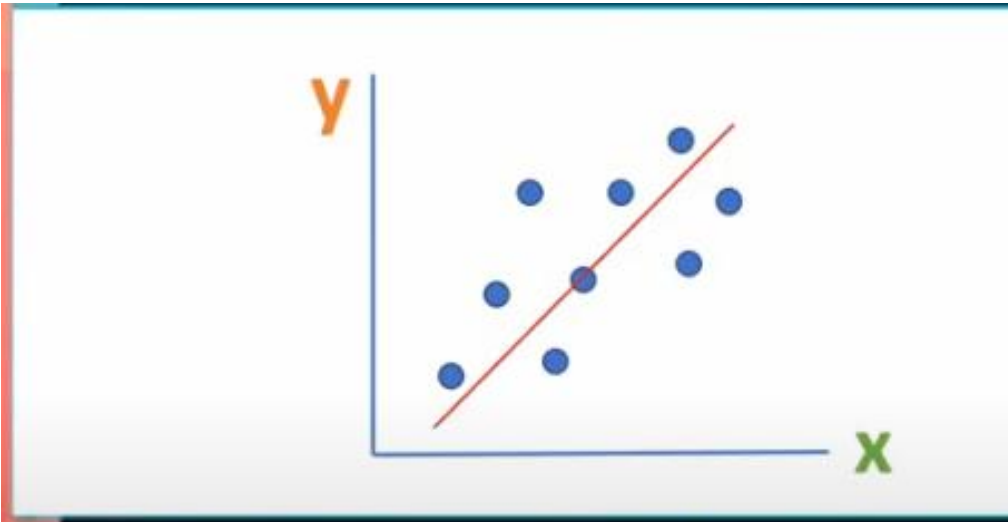
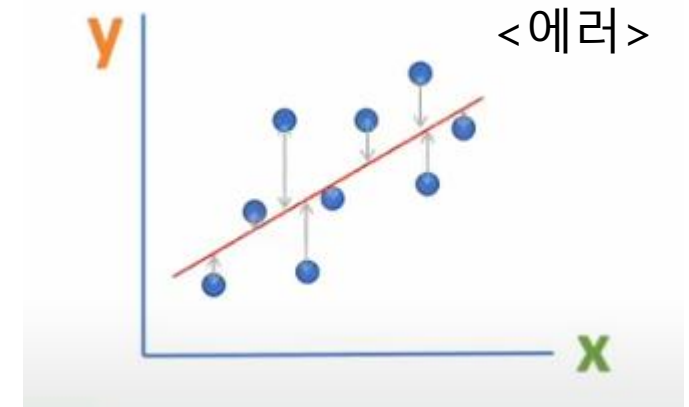
<학습: 오차의 면적이 축소되어 가는 것>



<데이터사이언스 모델링>

- 특징을 잘 설명하는 그래프를 찾아가는 과정
- 선형회귀: 1차 함수 모델링
- Gradient descent: 2차 함수 모델링

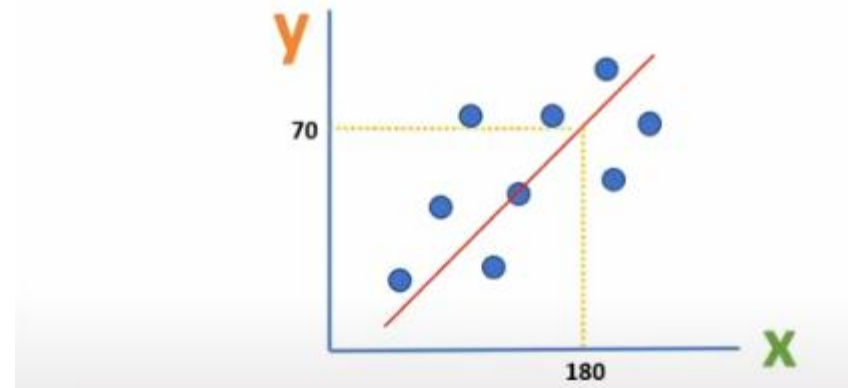
선형 회귀(Linear regression) 붉은 선을 찾아내는 방법



예) 키-180cm라면 몸무게 70kg 예측...그러나 에러(예측값과 실제값의 차이)가 있다.
붉은선은 최적의 선인가?

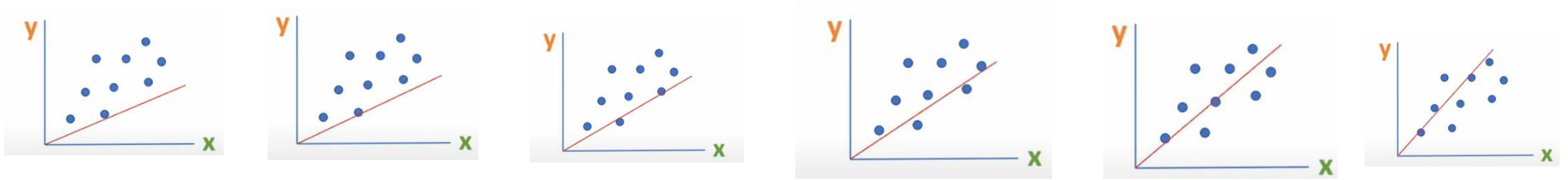
선형 회귀(Linear Regression)란?

- 파란 점들을 가장 잘 지나가는 '빨간 직선'을 찾는 방법
- x (키)를 알면 y (몸무게)를 예측하는 '최적의 선'을 찾는 과정
- 에러를 가장 작게 만들어주는 선이 바로 최적의 선(빨간 선)
- 수학적으로는, 모든 점들의 오차(거리의 제곱)를 더한 값이 최소가 되도록 선을 찾는 것



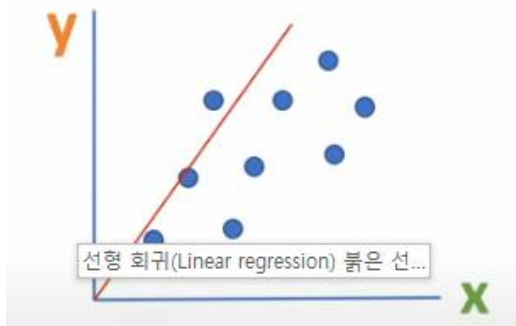
선형회기는 효율적이지 않다. 계산량이 많다.

<모든 기울기>

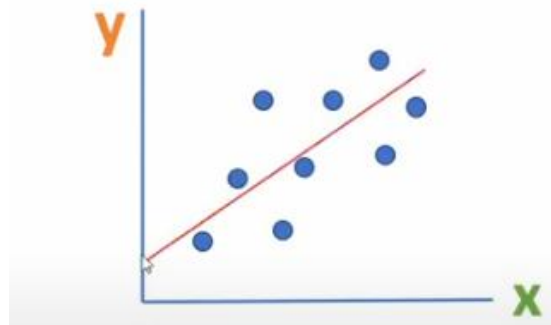


<y 절편, 인터셉트=바이어스>

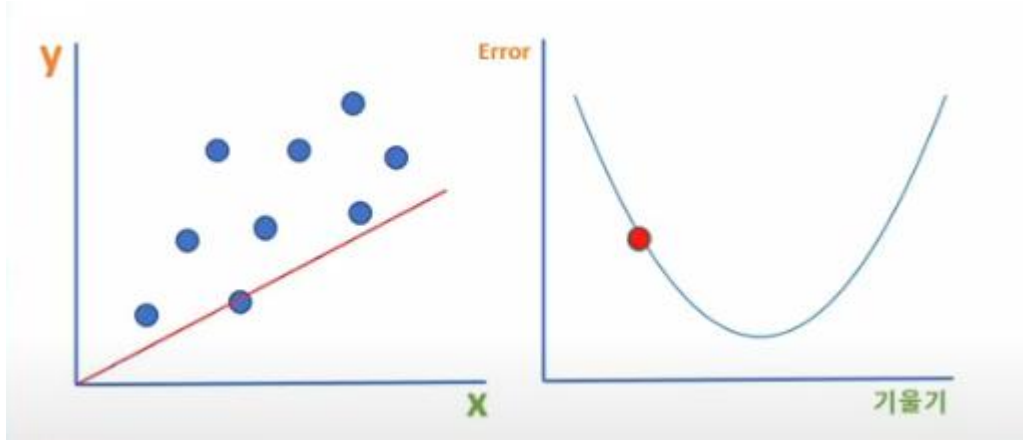
- 선형회귀는 '모든 직선 후보'를 다 비교해야 한다.
- 기울기뿐 아니라 'y 절편(bias)'도 무한히 바꿔 보아야 한다



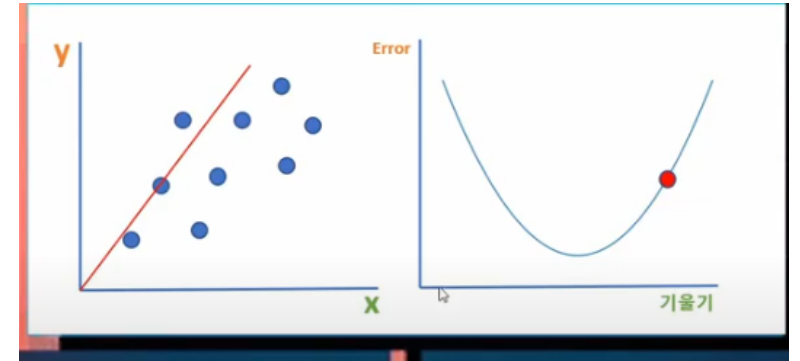
<y=0>



$Y > 0$



기울기 > 1...에러 크네.



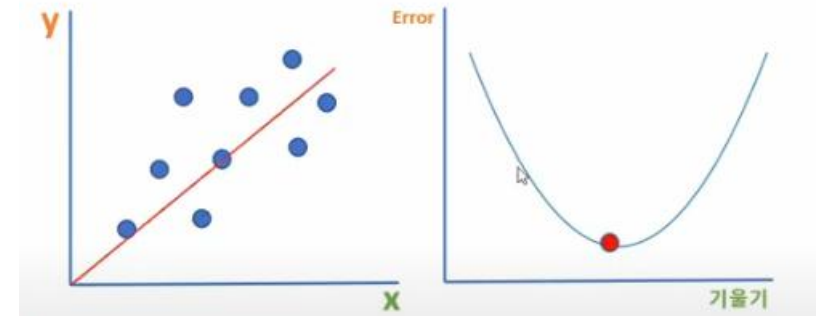
기울기 < 1...에러 크네.

- 에러: (예측값-실제값)의 거리
- 손실: 에러의 제곱
- 모든 기울기에 대한 에러값을 다 측정했을때...2차함수 그래프

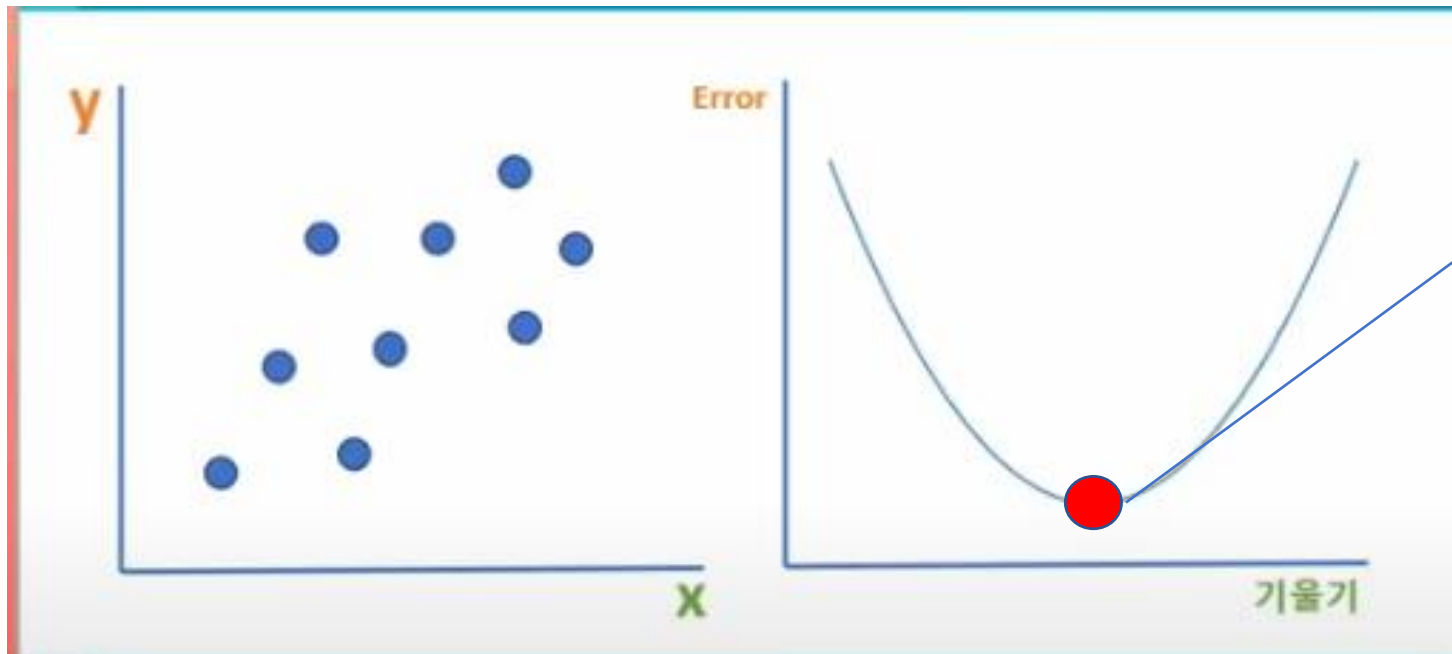
- $(-2)^2 = 4$
- $(-1)^2 = 1$
- $(0)^2 = 0$
- $(1)^2 = 1$
- $(2)^2 = 4$

즉,

- 양쪽으로 갈수록 값이 커지고
- 가운데(0 근처)에서 가장 작아짐.



기울기를 올렸더니...에러값이 작아졌다.

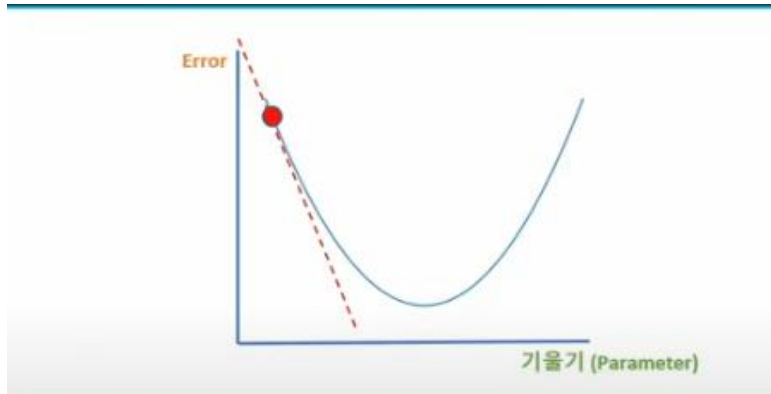


모든기울기의
에러값을 다 계
했을 때 최저값
여기....

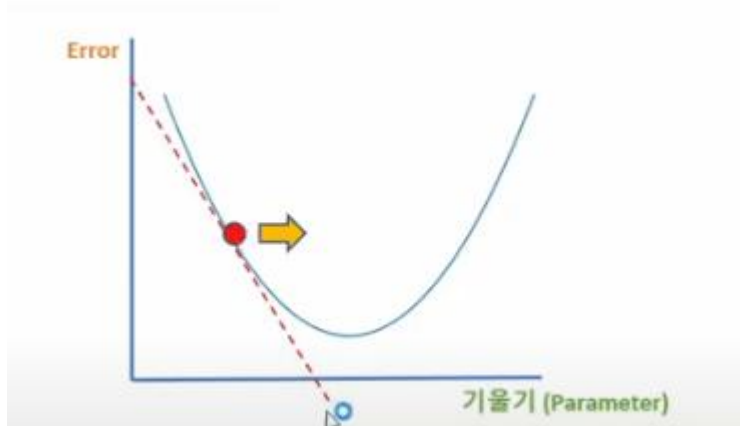
그러나

비효율적

효율적 계산 Gradient descent(경사하강)



<기울기 이동>



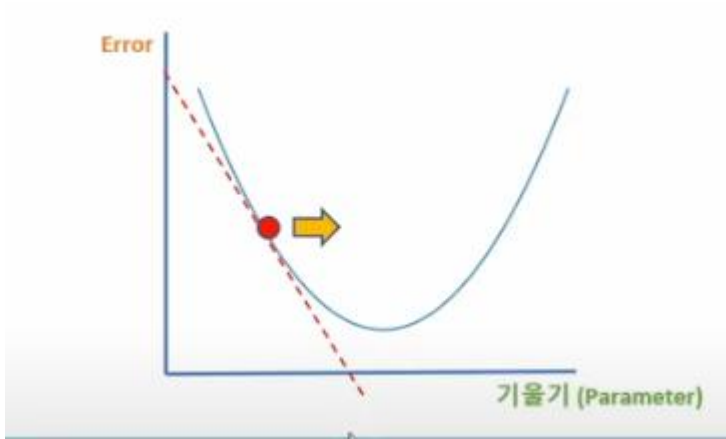
1. 2차 함수 한점의 미분값을 구한다.<접선의 기울기>
2. 이 접선의 기울기를 바탕으로 어느쪽으로 얼마만큼 이동할 것인지를 결정한다.

<방향>

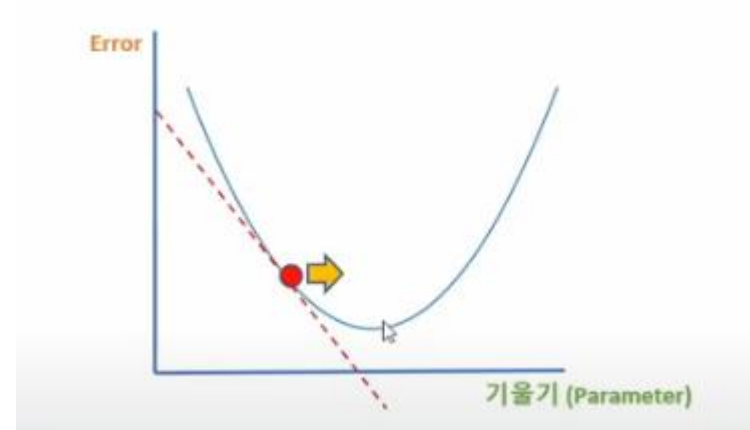
1. 기울기 < 0... 즉, 음수이면 오른쪽으로 이동
2. 기울기 > 0... 즉, 양수이면 왼쪽으로 이동

<이동 폭>

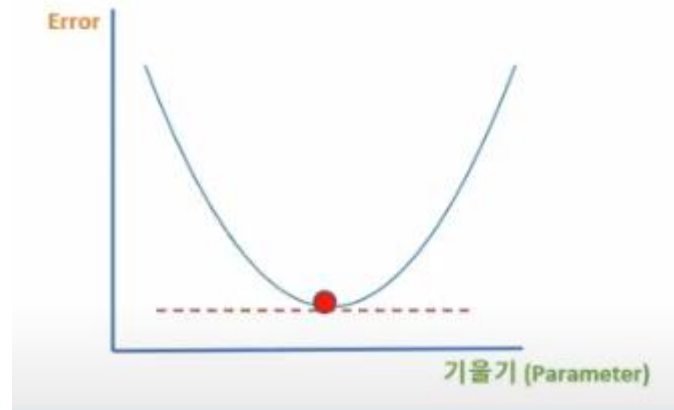
1. 기울기의 절대값이 크면...상대적으로 많이 이동
2. 기울기의 절대값이 작으면...상대적으로 적게 이동



1. 기울기 완만 but 여전히 음수



2. 점점 우측으로 이동...최소 에러값을 찾아서 이동



3. 마침내, 최소 에러값(기울기, 0)을 찾음

<장점>

1. 계산량이 적다.
2. 기울기가 클수록 많이 움직임
3. 기울기 작으면 미세하게 움직임



경사하강법의 작동 원리

- ① 손실함수
- ② 경사(Gradient, 기울기)
- ③ 학습률(Learning Rate)

- 경사하강법은 AI 모델이 스스로 틀린 것을 고쳐 나가는 과정이다.
- 경사는 '어느쪽으로 내려가야 더 빨리 낮아지는지 방향을 알려줌.'
- 손실함수는 '지금 내가 얼마나 틀렸는가?' 나타내는 숫자
- 학습률은 '한번에 얼마나 크게 발을 옮길지 정하는 크기 '

① 손실 함수(Loss Function)

모델이 얼마나 잘못 예측했는지 측정하는 함수입니다. 경사하강법이 최소화하려는 대상입니다.

$$J(W, b) = \frac{1}{2m} \sum_{i=1}^m (h_W(x^{(i)}) - y^{(i)})^2$$

(위는 평균 제곱 오차(MSE)의 예시이며, W 와 b 는 모델의 파라미터, $h_W(x)$ 는 예측값, y 는 실제값입니다.)

② 경사(Gradient, 기울기)

손실 함수를 최소화하기 위해 이동해야 할 방향을 알려줍니다.

- **경사(Gradient):** 손실 함수 $J(W, b)$ 를 각 파라미터(예: W)에 대해 미분한 값입니다.

$$\nabla J(W, b) = \frac{\partial J(W, b)}{\partial W}$$

- **기울기의 역할:** 미분 값은 특정 지점에서의 접선의 기울기를 나타냅니다. 이 기울기의 **부호**는 해당 파라미터 값을 증가시켜야 손실이 증가하는지 감소하는지 알려줍니다.
 - 기울기가 양수(+)이면: W 를 감소시켜야 J 가 감소합니다.
 - 기울기가 음수(-)이면: W 를 증가시켜야 J 가 감소합니다.
- **하강(Descent):** 경사하강법은 손실을 **최소화**해야 하므로, 기울기의 반대 방향($-\nabla J$)으로 파라미터를 이동시킵니다.

③ 학습률(Learning Rate, α)

파라미터를 업데이트할 때 한 번에 얼마나 멀리 이동할지 결정하는 하이퍼파라미터입니다.

- **학습률이 너무 크면:** 최솟값을 건너뛰고 발산할 수 있습니다.
- **학습률이 너무 작으면:** 수렴하는 데 시간이 너무 오래 걸릴 수 있습니다.

파라미터 업데이트 공식

경사하강법은 다음 공식을 사용하여 각 파라미터(W)를 반복적으로 업데이트합니다.

$$\text{New } W = \text{Old } W - \alpha \cdot \frac{\partial J(W, b)}{\partial W}$$

1. 현재 파라미터(W)에서 손실 함수의 기울기를 계산합니다. ($\frac{\partial J(W, b)}{\partial W}$)
2. 계산된 기울기에 학습률(α)을 곱합니다. (이동할 거리 결정)
3. 기울기의 반대 방향으로 파라미터 값을 업데이트합니다. (손실이 감소하는 방향으로 이동)

이 과정을 손실 함수가 최소값에 도달하거나, 업데이트 폭이 매우 작아질 때까지 반복합니다.

경사하강법의 종류

종류

확률적 경사하강법
(Stochastic Gradient
Descent, SGD)

배치 경사하강법 (Batch
Gradient Descent,
BGD)

미니배치 경사하강법
(Mini-Batch Gradient
Descent)

특징

한 번에 **하나의 데이터 샘플**에 대한 기울기만 계산하여 업데이트합니다.

한 번의 업데이트를 위해 **전체 데이터셋**의 기울기를 계산하여 평균 냅니다.

****미니 배치 크기(n)****만큼의 데이터 샘플에 대한 기울기를 계산하여 평균 냅니다. (가장 널리 사용됨)

장점

매우 빠름, 메모리 효율적, 지역 최솟값(local minimum) 탈출에 유리함.

안정적이고 정확한 최솟값 수렴을 보장함.

SGD의 속도와 BGD의 안정성을 결합. 계산 효율적이며 안정적으로 수렴함.

단점

업데이트 과정이 매우 불안정하고 진동이 심함.

데이터셋이 클 경우 계산 시간이 오래 걸리고 메모리 소모가 큼.

최적의 미니 배치 크기를 설정해야 함.

경사하강법의 개선

순수한 경사하강법은 최적화 과정에서 학습률을 고정하고 모든 파라미터에 동일하게 적용하는 등의 한계가 있습니다. 이를 개선하기 위해 다양한 고급 최적화 기법이 개발되었습니다.

- **모멘텀(Momentum):** 이동 경로에 '관성'을 부여하여 진동을 줄이고 빠르게 수렴하도록 돕습니다.
- **아다그라드(Adagrad):** 자주 등장하는 파라미터는 학습률을 감소시키고, 드물게 등장하는 파라미터는 학습률을 증가시켜 적응적으로 학습률을 조정합니다.
- **RMSProp:** Adagrad의 단점(학습률이 너무 빨리 감소함)을 보완하기 위해 가중치의 제곱을 지수 이동 평균으로 계산합니다.
- **아담(Adam, Adaptive Moment Estimation):** 모멘텀과 RMSProp의 장점을 결합한 최적화 기법으로, 현재 딥러닝에서 가장 많이 사용됩니다.



AdaGrad+RMSProp

AdaGrad와 RMSProp: 수식의 의미와 예시를 통한 이해

1. 배경: 경사 하강법(Gradient Descent)과 학습률

딥러닝 모델을 훈련하는 과정은 울퉁불퉁한 손실(Loss) 곡면 위에서 가장 낮은 지점(최소 손실)을 찾아 내려가는 것과 같습니다. 이 경로를 안내하는 것이 **경사 하강법**입니다.

경사 하강법에서 ****학습률(η , Learning Rate)****은 한 번에 얼마나 큰 보폭으로 내려갈지를 결정하는 값입니다.

- **전통적인 방법(SGD):** 모든 가중치(파라미터)와 모든 학습 단계에서 동일한 보폭(학습률)을 사용했습니다. 이는 비효율적이거나 불안정한 결과를 낼 수 있습니다.

2. AdaGrad (Adaptive Gradient)

AdaGrad는 **Adaptive Gradient**의 약자로, 학습률을 **파라미터별로 다르게 조정**하는 최초의 중요한 알고리즘 중 하나입니다.

- **핵심 아이디어:** '자주 변하는 파라미터는 조심스럽게(작은 보폭으로), 드물게 변하는 파라미터는 과감하게(큰 보폭으로) 업데이트하자.'
- **작동 방식:** 해당 파라미터의 과거 기울기 제곱 값을 모두 누적하여 학습률을 조정하며, 학습이 진행될수록 학습률을 점진적으로 감소시킵니다.

$$\begin{aligned}
 g_{t,i} &= \nabla J(\theta_i) \\
 G_{t,ii} &= \sum_{\tau=1}^t g_{\tau,i}^2 \\
 \theta_{t+1,i} &= \theta_{t,i} - \frac{\eta}{\sqrt{G_{t,ii} + \epsilon}} \cdot g_{t,i}
 \end{aligned}$$

1. $g_{t,i}$ (기울기): 모델이 현재 위치(θ_i)에서 내려가야 할 방향과 크기입니다.
2. $G_{t,ii}$ (누적 합): 해당 파라미터가 지금까지 얼마나 '움직였는지'를 기록하는 발자취 저장소입니다. 시간이 지날수록 이 값은 계속 커집니다.
3. $\frac{\eta}{\sqrt{G_{t,ii} + \epsilon}}$ (적응형 학습률): 이 부분이 보폭(학습률)을 조절합니다. 분모의 $G_{t,ii}$ 가 커질수록 전체 값은 작아져서 보폭이 줄어듭니다.

[예시: 학습률 급감 문제]

- 초기 학습률 (η): 0.01
- 매 스텝마다 기울기(g)가 0.1로 일정하게 발생한다고 가정합니다.

스텝(t)	누적 합 $G_{t,ii}$	적응형 학습률 ($\frac{0.01}{\sqrt{G_{t,ii}}}$)	결과
1	$0.1^2 = 0.01$	$\frac{0.01}{\sqrt{0.01}} = 0.1$	초기에는 큰 보폭
100	$100 \times 0.01 = 1.0$	$\frac{0.01}{\sqrt{1.0}} = 0.01$	보폭이 기본 학습률 수준으로 감소
1000	$1000 \times 0.01 = 10.0$	$\frac{0.01}{\sqrt{10.0}} \approx 0.003$	보폭이 매우 작아져 사실상 학습이 멈춥니다.

- 장점:** 희소한 특성(Sparse features)이 많은 문제(예: 자연어 처리)에서 큰 효과를 발휘하며, 드물게 등장하는 특성에 더 큰 업데이트 기회를 제공합니다.
- 치명적인 단점:** 기울기 제곱 값을 계속 누적($G_{t,ii}$) 하기 때문에, 학습이 오래 진행될수록 분모 값이 무한정 커져서 $\frac{\eta}{\sqrt{G_{t,ii} + \epsilon}}$ 항이 0에 가깝게 급격히 감소합니다. 결국 모델은 더 이상 학습하지 못하고 일찍 멈춰버리는 문제가 발생합니다.

3. RMSProp (Root Mean Square Propagation)

RMSProp은 AdaGrad의 치명적인 단점, 즉 학습률이 너무 빨리 0으로 수렴하는 문제를 해결하기 위해 등장했습니다.

- **핵심 아이디어:** AdaGrad처럼 파라미터별로 학습률을 조정하되, **과거의 모든 정보를 기억하지 말고 최근의 변화만 지수 이동 평균으로 반영**하자.
- **작동 방식:** 과거의 모든 기울기 정보를 누적하는 대신, ****지수 이동 평균(Exponentially Decaying Average)****을 사용하여 최근 기울기 정보에 더 큰 가중치를 부여하고, 오래된 정보는 점차적으로 잊어버립니다. 이를 통해 학습률이 과도하게 감소하는 것을 방지합니다.
- **수식:** 기울기 제곱의 지수 이동 평균 $E[g^2]_t$ 를 사용하여 파라미터 θ 를 갱신합니다.

$$E[g^2]_t = \gamma E[g^2]_{t-1} + (1 - \gamma)g_t^2$$
$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{E[g^2]_t + \epsilon}} \cdot g_t$$

(여기서 g_t 는 시점 t 에서의 기울기, γ 는 망각 인자(Decay Rate)로 보통 0.9가 사용되며, 최근 기울기의 반영 정도를 결정합니다.)

$$E[g^2]_t = \gamma E[g^2]_{t-1} + (1 - \gamma)g_t^2$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{E[g^2]_t + \epsilon}} \cdot g_t$$

RMSProp의 핵심은 기울기 제곱의 지수 이동 평균 $E[g^2]_t$ 입니다.

1. g_t (기울기): 모델이 현재 위치에서 내려가야 할 방향과 크기입니다.
2. $E[g^2]_t$ (이동 평균): AdaGrad의 $G_{\{t,ii\}}$ 와 달리, γ (0.9)와 $(1 - \gamma)$ (0.1)를 사용하여 과거 정보($E[g^2]_{t-1}$)는 0.9만큼 줄이고, 현재 정보(g_t^2)는 0.1만큼만 더합니다.
 - $\gamma E[g^2]_{t-1}$: 이전에 쌓였던 값에 망각 인자($\gamma < 1$)를 곱하여 값을 감소시킵니다.
 - $(1 - \gamma)g_t^2$: 현재 기울기 정보를 새로 반영합니다.
3. 결과: $E[g^2]_t$ 는 누적되지 않고 항상 일정한 범위 내에서 최근 기울기의 크기를 반영하게 되므로, 분모가 무한히 커지는 현상이 발생하지 않습니다.

🌸 심화 설명: 초기 편향(Bias) 문제와 불안정성

RMSProp은 지수 이동 평균을 사용하지만, 초기 단계에서는 이 평균값이 실제 평균보다 작게 편향되는 문제가 발생하며, 이로 인해 학습이 불안정해질 수 있습니다.

1. 문제의 원인 (수식 기반):

- 지수 이동 평균 $E[g^2]_t$ 는 보통 $E[g^2]_0 = 0$ 으로 초기화됩니다.
- γ 가 1에 가까운 값($\gamma = 0.9$)일수록, $(1 - \gamma)$ 는 작아집니다.
- 따라서 초기 $E[g^2]_t$ 는 실제 기울기 제곱의 평균을 $\sum_{\tau=1}^t (1 - \gamma)^t g_\tau^2$ 로 나타내는데, γ 의 지수 승으로 인해 실제 평균보다 **과소평가**됩니다.

2. 불안정성으로의 연결:

- $E[g^2]_t$ 가 과소평가되면, 업데이트 수식의 분모 $\sqrt{E[g^2]_t + \epsilon}$ 역시 작아집니다.
- 결과적으로, $\frac{\eta}{\sqrt{E[g^2]_t + \epsilon}}$ 항, 즉 ****실제 적용되는 학습률(보폭)****이 초기 단계에서 **과도하게 크게** 설정됩니다.

최적화 알고리즘	보폭(학습률) 조절 방법	비유
SGD	모든 파라미터에 동일한 보폭을 사용	눈을 감고 지도 전체에서 항상 똑같은 크기로 발을 내딛는 등산객
AdaGrad	파라미터별로 과거의 총 기울기 변화량을 기반으로 보폭 조정	각 방향(파라미터)으로 얼마나 자주, 얼마나 세게 발을 디뎠는지 모두 기록 해 두었다가, 기록이 많은 방향은 보폭을 아주 작게 줄이는 등산객
RMSProp	파라미터별로 최근 의 기울기 변화량을 기반으로 보폭 조정	각 방향(파라미터)으로 최근 에 발을 디딘 강도만 기억합니다. 덕분에 조심성은 유지하되, 초기에는 너무 과감하게 발을 내딛어 잠시 비틀거릴 수 있습니다.